

Escalonamento e Prioridade de Processos

Objetivo

Entender como o sistema operacional Linux decide qual processo deve utilizar o processador, e aprender a controlar a prioridade de processos usando as ferramentas nice e renice.

Pré-requisitos

- Docker instalado na máquina
- Docker Compose instalado

Como iniciar o laboratório

Suba o container:

```
docker compose up -d
```

Entre no container:

```
docker compose exec so-lab-03 bash --login
```

Conceito: O que é escalonamento?

O escalonador do Linux é a parte do sistema operacional responsável por decidir qual processo vai usar o processador em cada momento. Como vários processos competem pela CPU ao mesmo tempo, o escalonador usa critérios como prioridade e tempo de espera para tomar essa decisão.

A prioridade de um processo no Linux é controlada pelo valor nice:

- Varia de -20 (maior prioridade — processo é "menos gentil" com os outros) até 19 (menor prioridade — processo cede espaço para os demais)
- O valor padrão é 0
- Apenas root pode definir valores negativos (alta prioridade)

Importante: a diferença de prioridade só é visível quando os processos competem pelo mesmo núcleo de CPU. Por isso, este container está configurado para usar apenas 1 núcleo (cpuset: "0"), garantindo que a disputa aconteça e o impacto da prioridade seja claramente observado.

Exercícios

Exercício 1 — Verificando a prioridade dos processos

Liste todos os processos mostrando suas colunas de prioridade:

```
ps -eo pid,ni,pri,cmd --sort=ni
```

-eo define quais colunas exibir: pid, ni (valor nice), pri (prioridade interna do kernel), cmd (comando). --sort=ni ordena pelo valor nice.

Observe que a maioria dos processos tem NI = 0, que é o padrão.

Exercício 2 — Iniciando um processo com prioridade baixa

O comando nice permite iniciar um processo já com uma prioridade definida. Inicie um processo de carga com prioridade baixa (valor 15):

```
nice -n 15 stress-ng --cpu 1 --timeout 60s > /dev/null 2>&1 &
```

-n 15 define o valor nice como 15 (baixa prioridade). O & faz rodar em segundo plano. O > /dev/null 2>&1 redireciona a saída para lugar nenhum, evitando que mensagens poluam o terminal.

Verifique se o processo foi criado com a prioridade correta:

```
ps -eo pid,ni,cmd | grep stress
```

Observe o valor NI = 15 na saída.

Exercício 3 — Iniciando um processo com prioridade alta

Inicie outro processo de carga com prioridade alta (valor -10). Como é um valor negativo, precisa de root:

```
nice -n -10 stress-ng --cpu 1 --timeout 60s > /dev/null 2>&1 &
```

Liste os dois processos e compare:

```
ps -eo pid,ni,cmd | grep stress
```

Você deve ver dois processos stress-ng com valores NI diferentes: um com 15 e outro com -10.

Exercício 4 — Alterando a prioridade de um processo em execução

O `renice` permite mudar a prioridade de um processo que já está rodando. Primeiro, crie um processo:

```
stress-ng --cpu 1 --timeout 120s > /dev/null 2>&1 &
```

Pegue o PID dele:

```
ps -eo pid,ni,cmd | grep stress
```

Agora altere a prioridade para 10:

```
renice -n 10 -p
```

Substitua <PID> pelo número do processo. -n 10 define o novo valor nice e -p especifica o PID alvo.

Verifique se a mudança foi aplicada:

```
ps -eo pid,ni,cmd | grep stress
```

Exercício 5 — Observando o impacto no htop

Com processos de diferentes prioridades competindo pelo mesmo núcleo de CPU, o impacto da prioridade fica visível. Inicie os três processos e abra o `htop`:

```
stress-ng --cpu 1 --timeout 60s > /dev/null 2>&1 &  
nice -n 19 stress-ng --cpu 1 --timeout 60s > /dev/null 2>&1 &  
nice -n -10 stress-ng --cpu 1 --timeout 60s > /dev/null 2>&1 &  
htop
```

No `htop`, pressione F6 e ordene por NI para ver os processos organizados por prioridade. Observe como o processo com NI = -10 recebe mais tempo de CPU do que o com NI = 19 — pois estão disputando o mesmo núcleo.

Por que isso acontece? Quando a máquina tem múltiplos núcleos livres, cada processo ocupa um núcleo diferente e todos recebem 100% — a prioridade não faz diferença. Com apenas 1 núcleo disponível, o escalonador precisa dividir o tempo entre os processos, e é aí que a prioridade importa. Regra prática: quanto menor o valor nice, maior a fatia de CPU que o processo recebe. Um processo com NI = -10 vai receber muito mais tempo de processador do que um com NI = 19 disputando o mesmo núcleo.

Pressione q para sair do `htop`.

Exercício 6 — Encerrando todos os processos stress-ng

Ao final, encerre todos os processos de carga de uma vez:

```
pkill stress-ng
```

pkill encerra todos os processos com o nome especificado.

Como encerrar o laboratório

Saia do container:

```
exit
```

Desligue o container:

```
docker compose down
```